

1-1 · 리눅스 커맨드 기초 · 실습 1H

텍스트 처리와 파이프라인

작은 명령을 | 로 이어 로그에서 숫자를 뽑는다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

① `grep` 의 `-c` · `-i` · `-v` · `-n` 으로 원하는 줄만 골라낸다

② `sort | uniq -c | sort -rn` 을 한 덩어리로 쓴다

③ `cut -d ' ' -f N` 으로 칸을 잘라 집계한다

④ `>` · `>>` · `tee` 로 결과를 파일에 남긴다

실무 로그는 수십만 줄 — 눈으로 볼 수 없어 걸러 세어야 하고, 그 도구가 `grep·sort·uniq·wc`

같은 access.log 하나를 여덟 단계로 여러 각도에서 본다

새 파일을 만들지 않습니다 — 한 데이터를 여러 각도로 보는 감각이 이 과정 내내 쓰입니다

로그 만들기

히어독으로 8줄 · wc -l
이 8

grep 으로 고르기

-c · -i · -v · -n

파이프로 잇기

sort | uniq -c | sort
-rn

자르고 앞뒤 보기

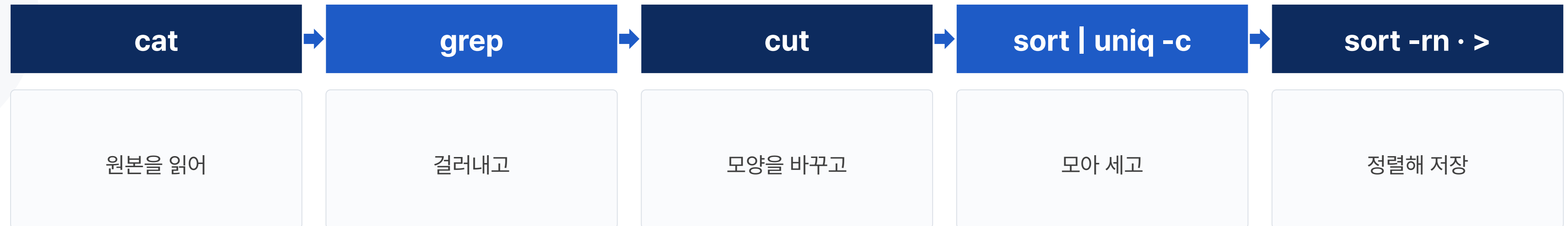
cut · head · tail

남기고 키우기

> · >> · tee · 800줄

1. 파이프

앞 명령의 출력이 곧바로 뒤 명령의 입력으로 흐른다



아주 작은 데이터 파이프라인 — 5주차 Spark 도 규모만 다를 뿐 순서는 똑같다

| 는 다음 명령에 넘기고 > 는 파일에 쓴다 — | 는 몇 번이든, > 는 보통 맨 끝 한 번

grep — 원하는 줄만, 반대로, 겹쳐서, 앞뒤 맥락까지

명령	뜻	확인
grep ERROR access.log	ERROR 가 든 줄만	3줄
grep -c ERROR	개수만	3
grep -i error	대소문자 무시	3줄
grep -v ERROR	반대로 — 아닌 줄만	5줄 (3 + 5 = 8)
grep -n ERROR	줄 번호까지	번호가 앞에
grep -E 'ERROR WARN'	또는(OR) — -E 가 를 「또는」으로	4줄
grep INFO ... grep kim	그리고(AND) — 파이프로	kim 의 2줄
grep -B 1 -A 1 WARN	앞 1줄 · 뒤 1줄 맥락	WARN 가운데 3줄

grep 에 AND 옵션은 없다 — 파이프로 이어서 거른다. 오류는 그 줄만 보면 원인을 못 찾으니 -A · -B 를 자주 쓴다

uniq 는 이웃한 중복만 뭉는다 — sort 를 빼면 집계가 틀린다

sort 없이 uniq -c 만

떨어져 있는 같은 줄이 따로따로 세어진다

- ERROR db timeout 이 세 군데 흩어져 있다
- 이웃하지 않으니 각각 1 로 센다
- 실무에서 집계가 이상하게 나오는 흔한 원인

VS

sort | uniq -c | sort -rn

한 덩어리로 외워 둔다 — 첫 줄이 3 ERROR db timeout

- sort — 같은 줄을 이웃하게 모은다
- uniq -c — 묶으면서 개수를 앞에
- sort -rn — 숫자(n) 기준 내림차순(r)

cut 으로 칸을 잘라 집계 — 미니 과제까지

하려는 일	명령	확인
앞 3줄 / 뒤 2줄	head -n 3 · tail -n 2	—
두 번째 칸만	cut -d ' ' -f 2 access.log	-d 구분자 · -f 몇 번째
레벨(첫 칸) 집계	cut -d ' ' -f 1 ... sort uniq -c sort -rn	INFO 4 · ERROR 3 · WARN 1
사용자별 접속 횟수	grep 'user=' ... cut -d ' ' -f 3 sort uniq -c	2 user=kim 이 맨 위
순서 고정	... sort -k1,1rn -k2,2	개수 같으면 이름순

긴 파이프라인은 | 앞까지만 먼저 실행해 결과를 보고 한 조각씩 붙인다 — 어디서 틀렸는지 바로 보인다

결과를 파일로 남기고, 파일이 커져도 명령은 그대로

1 > 덮어쓰기 · >> 이어 붙이기
grep ERROR > errors.txt(3줄) → grep WARN >> errors.txt(4줄)

2 tee — 화면에도, 파일에도
... | sort -rn | tee level_count.txt

3 800줄로 키워도 명령은 그대로
access.log 를 100번 이어 붙인 big.log 도 같은 명령 → INFO 400 · ERROR 300 · WARN 100

원본 보호

결과는 항상
다른 이름으로

for i in \$(seq 1 100); do cat access.log; done > big.log — \$() 는 괄호 안(seq 1 100 → 1...100)을 먼저 실행해 그 자리에 넣는다

숫자가 이렇게 나오면 성공

- 1 `wc -l access.log` 가 8 — 다르면 붙여넣기가 잘린 것, 지우고 다시
- 2 `grep -c ERROR` 가 3, `grep -v ERROR` 가 5줄 ($3 + 5 = 8$)
- 3 `sort | uniq -c | sort -rn` 첫 줄이 3 ERROR db timeout
- 4 `cut -f 1` 집계가 INFO 4 · ERROR 3 · WARN 1
- 5 `big.log` 가 800줄, 집계가 정확히 100배

더 풀어 보기 — 가장 많은 오류 한 줄(`head -n 1`) · user= 없는 줄 수(`-vc → 4`) · 이름만(`sort -u`)

1-1 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

오늘 쓴 명령을 한 장으로 모으고, 자주 겪는 오류 세 가지를 미리 짚습니다

✓ 오늘의 커맨드 요약표

✓ 체크리스트 6항목 점검

✓ command not found · Permission denied · No such file

✓ 예상 질문과 1-2 예고

준비할 것

errors.txt · level_count.txt 가 남은 상태